# AN ENGINEERING PROJECT PROGRESS REPORT

**On**

# SyncWrite - A collaborative document writing platform


**Submitted By**

**Aayush Karki - 200101**

**Anish Bista - 200105**

**Rabin Mishra - 200128**

**Pragya Pokharel - 200123**

**Pankaj Kumar Yadav - 200121**


**Submitted to**

**The Department of IT and Computer Engineering**

**In Partial fulfillment of requirement for the degree of**

**Bachelor of Engineering in Information Technology**



**Cosmos College of Management & Technology**

**(Affiliated to Pokhara University)**

**Tutepani Lalitpur, Nepal**


**2082/04/7**

# DECLARATION

We hereby declare that the report of the project entitled "**SyncWrite - A collaborative document writing platform**" which is being submitted to the Department of ICT, Cosmos College of Management and Technology, Tutepani-14, Lalitpur in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering in IT Engineering, is a Bonafide report of the work carried out by us. The materials contained in this report have not been submitted to any University or Institution for the award of any degree and we are the only author of this complete work and no sources other than the listed here have been used in this word.

**Aayush Karki - 200101**
**Anish Bista - 200105**
**Rabin Misha - 200128**
**Praagya Pokharel - 200123**
**Pankaj Kumar Yadav - 200121**

## APPROVED BY

……..

**Prof. Dr. Ram Kumar Sharma**

Principal

Cosmos College of Management & Technology

# CERTIFICATE OF APPROVAL

The project report entitled **"SyncWrite - A collaborative document writing platform"**, submitted by Aayush Karki, Anish Bista, Rabin Mishra, Praagya Pokharel and Pankaj Kumar Yadav, in partial fulfillment of the requirement for the Bachelor's degree in IT Engineering has been accepted as a bona fide record of work independently carried out by the group in the department.

……..
**External Examiner**
Er. Akkal Bist
Head Technical Assistant-IT
Tribhuvan University

……
**Project Supervisor**
Er. Ajaya Adhikari

# COPYRIGHT

# ACKNOWLEDGEMENT

We would like to express our sincere gratitude to **Cosmos College of Management and Technology** for providing us the opportunity to work on this major engineering project titled **"SyncWrite: A Collaborative Document Writing Platform"**. This project represents the culmination of our academic learning and technical growth throughout our undergraduate journey.

We are deeply thankful to **Mr. Ajaya Adhikari**, Head of the Department of Information and Communication Technology, for his consistent support, guidance, and vision throughout the proposal and planning phases of this project. His insights into full-stack development, real-time systems, and modern software architecture have been instrumental in shaping the scope and direction of our work.

We would also like to extend our heartfelt appreciation to **Mr. Chiranjibi Pandey**, Deputy Head of the ICT Department, for his continuous encouragement and belief in our technical potential. His emphasis on hands-on, project-based learning has motivated us to take on a practical and impactful challenge through this collaborative document writing platform.

We also extend our gratitude to the faculty members of the Department of Information and Communication Technology at Cosmos College for their foundational teaching in Computer Science, Software Engineering, and Database Systems, which has equipped us with the confidence and skills to undertake this project. Additionally, we acknowledge the valuable feedback, discussions, and teamwork from our peers, which have greatly contributed to the development of this platform. This project reflects not only our technical skills but also the spirit of collaboration and innovation fostered at Cosmos.

We are grateful for this opportunity to design and build a scalable, real-world collaborative system as part of our academic journey, and we look forward to contributing meaningfully through this project.

**Aayush Karki**
**Anish Bista**
**Rabin Mishra**
**Praagya Pokharel**
**Pankaj Kumar Yadav**

# ABSTRACT

This report details the design, development, and implementation of **SyncWrite**, a collaborative document writing platform designed to facilitate real-time co-authoring of rich text documents across multiple devices. Leveraging **Flutter** for a cross-platform frontend and **Node.js** as a run time with **Express.JS** as a backend framework **with MongoDB** for a robust backend, SyncWrite provides a seamless user experience for document creation, editing, and sharing. Key features include secure user authentication via Google Sign-In (without Firebase), dynamic routing, a rich text editor powered by Flutter Quill, and real-time synchronization of document changes through **Socket.IO.** The system incorporates an **auto-save** mechanism and persistent user sessions, addressing common challenges in collaborative editing environments. This project demonstrates a comprehensive full-stack solution for modern web and mobile productivity applications.

# TABLE OF CONTENT

**Title** **Page**

# LIST OF FIGURES

# LIST OF ACRONYMS

| Acronym | Full form |
|---|---|
| API | Application Programming Interface |
| CRUD | Create, Read, Update, Delete |
| DB | Database |
| DBMS | Database Management System |
| IDE | Integrated Development Environment |
| MVVM | Model View View Model |
| OOP | Object Oriented Programming |
| REST | Representational State Transfer |
| SDK | Software Development Kit |
| UI | User Interface |
| UX | User Experience |
| VCS | Version Control System |

# DECLARATION

We hereby declare that the report of the project entitled "**SyncWrite - A collaborative document writing platform**" which is being submitted to the Department of ICT, Cosmos College of Management and Technology, Tutepani-14, Lalitpur in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering in IT Engineering, is a Bonafide report of the work carried out by us. The materials contained in this report have not been submitted to any University or Institution for the award of any degree and we are the only author of this complete work and no sources other than the listed here have been used in this word.

**Aayush Karki - 200101**

**Anish Bista - 200105**

**Rabin Misha - 200128**

**Praagya Pokharel - 200123**

**Pankaj Kumar Yadav - 200121**

# CERTIFICATE OF APPROVAL

The project report entitled **"SyncWrite - A collaborative document writing platform"**, submitted by Aayush Karki, Anish Bista, Rabin Mishra, Praagya Pokharel and Pankaj Kumar Yadav, in partial fulfillment of the requirement for the Bachelor's degree in IT Engineering has been accepted as a bona fide record of work independently carried out by the group in the department.

……..
**External Examiner**
Er. Akkal Bist
Head Technical Assistant-IT
Tribhuvan University

……
**Project Supervisor**
Er. Ajaya Adhikari

# COPYRIGHT

# ACKNOWLEDGEMENT

We would like to express our sincere gratitude to **Cosmos College of Management and Technology** for providing us the opportunity to work on this major engineering project titled **"SyncWrite: A Collaborative Document Writing Platform"**. This project represents the culmination of our academic learning and technical growth throughout our undergraduate journey.

We are deeply thankful to **Mr. Ajaya Adhikari**, Head of the Department of Information and Communication Technology, for his consistent support, guidance, and vision throughout the proposal and planning phases of this project. His insights into full-stack development, real-time systems, and modern software architecture have been instrumental in shaping the scope and direction of our work.

We would also like to extend our heartfelt appreciation to **Mr. Chiranjibi Pandey**, Deputy Head of the ICT Department, for his continuous encouragement and belief in our technical potential. His emphasis on hands-on, project-based learning has motivated us to take on a practical and impactful challenge through this collaborative document writing platform.

We also extend our gratitude to the faculty members of the Department of Information and Communication Technology at Cosmos College for their foundational teaching in Computer Science, Software Engineering, and Database Systems, which has equipped us with the confidence and skills to undertake this project. Additionally, we acknowledge the valuable feedback, discussions, and teamwork from our peers, which have greatly contributed to the development of this platform. This project reflects not only our technical skills but also the spirit of collaboration and innovation fostered at Cosmos.

We are grateful for this opportunity to design and build a scalable, real-world collaborative system as part of our academic journey, and we look forward to contributing meaningfully through this project.

**Aayush Karki**
**Anish Bista**
**Rabin Mishra**
**Praagya Pokharel**
**Pankaj Kumar Yadav**

# CHAPTER 1

# INTRODUCTION

## 1.1 Project Overview

SyncWrite is a contemporary collaborative document writing platform that aims to replicate the core functionalities of popular tools like Google Docs, providing users with a fluid and efficient environment for creating and editing documents together in real-time. The project addresses the growing demand for flexible and accessible collaborative tools in academic, professional, and personal contexts.

## 1.2 Motivation

Traditional document creation often involves sequential editing, version control complexities, and communication overhead. The motivation behind SyncWrite stems from the need for a real-time, simultaneous editing solution that enhances productivity, reduces friction in collaborative workflows, and provides a rich editing experience. Our goal was to build a multi-platform application from scratch, gaining deep insights into full-stack development, real-time communication protocols, and robust authentication mechanisms.

## 1.3 Problem Statement

The primary problems addressed by SyncWrite include:

- **Inefficient Collaborative Workflows:** Current methods often involve sending documents back and forth, leading to version conflicts and delays.
- **Lack of Real-time Synchronization:** Many basic editors lack the ability for multiple users to contribute simultaneously and see changes instantly.
- **Platform Dependency:** Users often require specific software or operating systems, limiting accessibility.
- **Complex Authentication:** Implementing secure and user-friendly authentication without relying solely on third-party services like Firebase can be challenging.

### 1.4 Objective & Application

The main objectives of the SyncWrite project were to:

- Develop a cross-platform (Web, Android, iOS) collaborative document editor using Flutter for the frontend and Node.js for the backend.
- Implement secure user authentication using Google Sign-In with Google Cloud OAuth, avoiding Firebase.
- Create RESTful APIs for user management and document CRUD (Create, Read, Update, Delete) operations.
- Integrate a rich text editor capable of handling various formatting options (bold, italic, bullets, code snippets).
- Enable real-time collaboration between multiple users on the same document using Socket.IO.
- Implement an efficient auto-save functionality to prevent data loss.
- Ensure persistent user sessions through JWT tokens and local storage.
- Provide a user-friendly interface for document listing, creation, and sharing.

# CHAPTER 2
# LITERATURE REVIEW

The development of SyncWrite draws upon established patterns and technologies in full-stack application development, real-time communication, and rich text editing.

- **Flutter for Cross-Platform UI:** Flutter, developed by Google, has emerged as a leading framework for building natively compiled applications for mobile, web, and desktop from a single codebase. Its expressive UI toolkit and performance benefits were key factors in its selection for SyncWrite's frontend.
- **Node.js and Express for Backend:** Node.js provides a highly scalable and efficient environment for backend services, especially when dealing with I/O-bound operations. Express.js, a minimalist web framework for Node.js, facilitates the creation of robust RESTful APIs.
- **MongoDB for NoSQL Database:** MongoDB, a NoSQL database, offers flexibility in schema design and scalability, making it suitable for storing dynamic content like rich text documents and user data. Mongoose, an ODM (Object Data Modeling) library for MongoDB and Node.js, simplifies data interactions.
- **Socket.IO for Real-time Communication:** Socket.IO enables low-latency, bidirectional, event-based communication between the client and server. It provides robust real-time capabilities crucial for features like collaborative editing, where instantaneous updates are paramount.
- **Quill for Rich Text Editing:** Quill is a powerful open-source rich text editor. Its delta format, a highly expressive JSON-based format for representing document content and changes, is ideal for collaborative environments as it simplifies the merging and synchronization of concurrent edits. Flutter Quill provides a robust integration of Quill capabilities within Flutter applications.
- **OAuth 2.0 for Authentication:** OAuth 2.0 is the industry-standard protocol for authorization. Implementing Google Sign-In via Google Cloud OAuth provides a secure and widely adopted method for user authentication without requiring a separate user credential system, enhancing user experience and security.
- **JWT for Session Management:** JSON Web Tokens (JWTs) are a compact, URL-safe means of representing claims to be transferred between two parties. They are widely

used for stateless authentication and session management in modern web applications, providing a secure way to maintain user login status.

Previous projects often faced challenges related to fragmented features, complex setup for real-time synchronization, and limited cross-platform support. SyncWrite aims to integrate these components into a cohesive, user-friendly, and performant platform.

# CHAPTER 3
# REQUIREMENTS & ANALYSIS

## 3.1 Functional Requirements

SyncWrite fulfills the following core functional requirements:

- **User Authentication:** Users must be able to sign in and sign up using their Google accounts.3
- **Document Management:** Users can create, open, edit, and delete documents.
- **Rich Text Editing:** The platform must support various text formatting options (bold, italic, underline, lists, code blocks, etc).
- **Real-time Collaboration:** Multiple users can simultaneously edit the same document, with changes appearing instantly for all collaborators.
- **Auto-Save:** Document changes are automatically saved periodically to the server.
- **Document Sharing:** Users can share documents with others via a shareable link.
- **Document Listing:** A dashboard or list view displays all documents created or shared with the user.
- **Persistent Sessions:** Users remain logged in across sessions using secure tokens.

## 3.2 Non-Functional Requirements

1. **Usability:** Intuitive and user-friendly interface across all supported platforms (web, Android, iOS).
2. **Performance:** Real-time updates should have minimal latency. Document loading and saving should be fast.
3. **Scalability:** The backend should be designed to handle a growing number of users and concurrent document editing sessions.
4. **Security:** User data and document content must be secure. Authentication tokens should be protected. Role-based access (if implemented for sharing permissions) should be enforced.

5. **Reliability:** The system should be robust and available, even under moderate load.
6. **Maintainability:** The codebase should be modular, well-documented, and easy to extend or debug.
7. **Portability:** The Flutter frontend ensures portability across target platforms (Web, Android, iOS), and the Dockerized backend facilitates deployment.

### 3.3 Technical Requirements

- **Frontend Language/Framework:** Dart/Flutter
- **Backend Language/Runtime:** JavaScript/Node.js
- **Backend Framework:** Express.js
- **Database:** MongoDB
- **Database ODM:** Mongoose
- **Real-time Communication:** Socket.IO
- **Authentication:** Google Cloud OAuth 2.0, JWT
- **Rich Text Editor:** Flutter Quill
- **Routing (Flutter):** RouteMaster
- **State Management (Flutter):** Riverpod
- **Version Control:** Git/GitHub

### 3.4 Feasibility Analysis

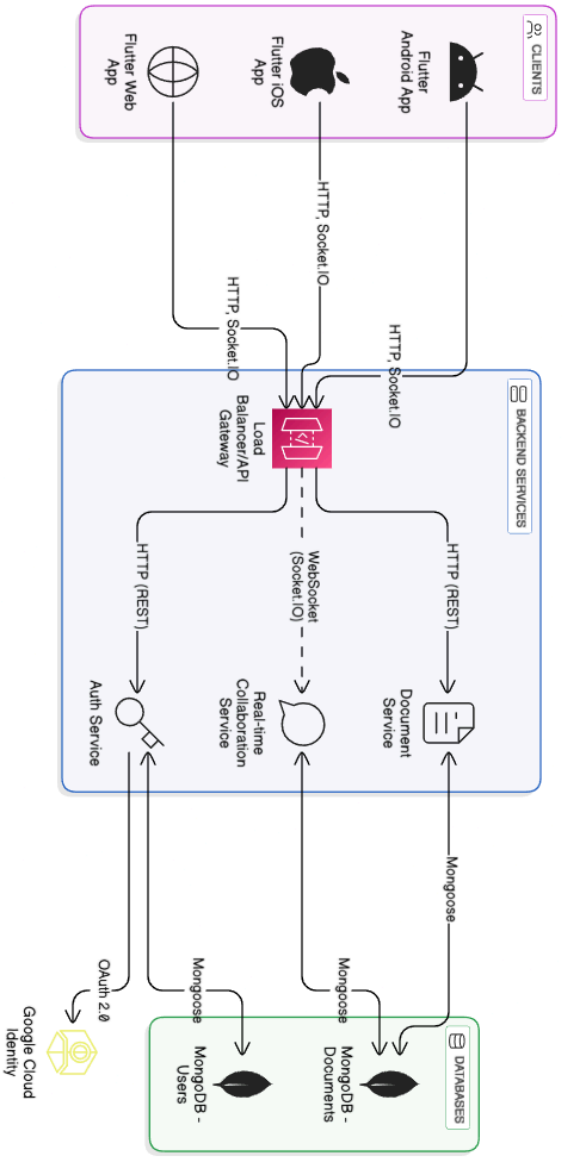
- **Technical Feasibility:** All chosen technologies (Flutter, Node.js, MongoDB, Socket.IO) are mature, well-documented, and have strong community support, making the implementation technically feasible.
- **Operational Feasibility:** The system's design promotes ease of use for end-users and straightforward management. The collaborative nature directly addresses operational needs for team-based work.
- **Economic Feasibility:** The use of open-source technologies (Flutter, Node.js, MongoDB) significantly reduces development and deployment costs.
- **Schedule Feasibility:** The project timeline, though ambitious for a full-stack, real-time application, was deemed feasible through modular development and iterative sprints, as outlined in the project plan.

# CHAPTER 4

# SYSTEM ARCHITECTURE & DESIGN

The SyncWrite platform employs a modern, layered architecture consisting of a Flutter-based frontend, a Node.js Express backend, and a MongoDB database, with real-time communication facilitated by SOCKET.IO

## 4.1 System Architecture



### 4.2 Component Breakdown

1. Client Applications (Flutter):
    - Developed using Flutter and Dart, ensuring a consistent UI/UX across web, Android, and iOS.
    - Manages user interaction, displays the rich text editor, document lists, and handles user authentication flow.
    - Utilizes **google_sign_in** for Google OAuth integration.
    - Manages application state using Riverpod.
    - Handles client-side routing with RouteMaster for dynamic and protected routes.
    - Communicates with the backend via RESTful HTTP requests for user and document management, and via WebSockets (Socket.IO) for real-time updates.
2. Backend Services (Node.js/Express):
    - Authentication Service: Handles user registration and login. Integrates with Google Cloud Identity for OAuth 2.0 authentication. Upon successful authentication, generates and issues JWTs for secure session management.
    - Document Service: Manages CRUD operations for documents. This includes creating new documents, retrieving existing ones, updating document metadata (e.g., title), and deleting documents.
    - Real-time Collaboration Service (Socket.IO): This is the core of real-time editing. It manages WebSocket connections, broadcasts document changes to all clients connected to a specific document "room," and handles the auto-save mechanism. It receives Quill's delta format changes, processes them, and relays them.
3. Database (MongoDB):
    - Stores user information (e.g googleId, name, email, profilePic) and document data.
    - Document content is stored in Quill's delta format, allowing for efficient representation and manipulation of rich text.
    - Mongoose is used as an ODM to define schemas for users and documents, simplifying interactions with MongoDB from the Node.js backend.

4. External Services:
    - Google Cloud Identity: Provides the OAuth 2.0 endpoint for secure Google Sign-In, issuing authentication tokens after user consent.

### 4.3 Document Schema

The Document schema is designed to store the content and metadata of each collaborative document.

- **_id**: ObjectId: Similar to the User schema, this is the unique identifier for each document, automatically generated by MongoDB.
- **title**: String: Stores the human-readable title of the document. This is important for user organization and identification in document lists.
- **content**: Object (Quill Delta format): This is the most critical field for the collaborative editor. It stores the actual rich text content of the document. The Quill Delta format is a lightweight, JSON-based representation of document changes, rather than the entire document's HTML or plain text. This format is highly efficient for real-time collaboration because it allows for easy composition and reconciliation of changes from multiple users, minimizing data transfer and simplifying conflict resolution.
- **createdAt**: Date: Records the timestamp when the document was initially created. Useful for sorting and display.
- **updatedAt**: Date: Records the timestamp of the last modification to the document. This is crucial for tracking recent activity and for auto-save functionality.
- **uid:** String (User ID of the creator): Stores the _id of the user who created the document, establishing a clear ownership link.
- **array of sharedWith: [{ userId: String, permission: String }]** for sharing: This commented-out section suggests a future or optional extension for advanced sharing features. If implemented, this array would store references to other users (userId) who have access to the document, along with their assigned permission levels (e.g., 'viewer', 'editor'), enabling controlled collaboration.

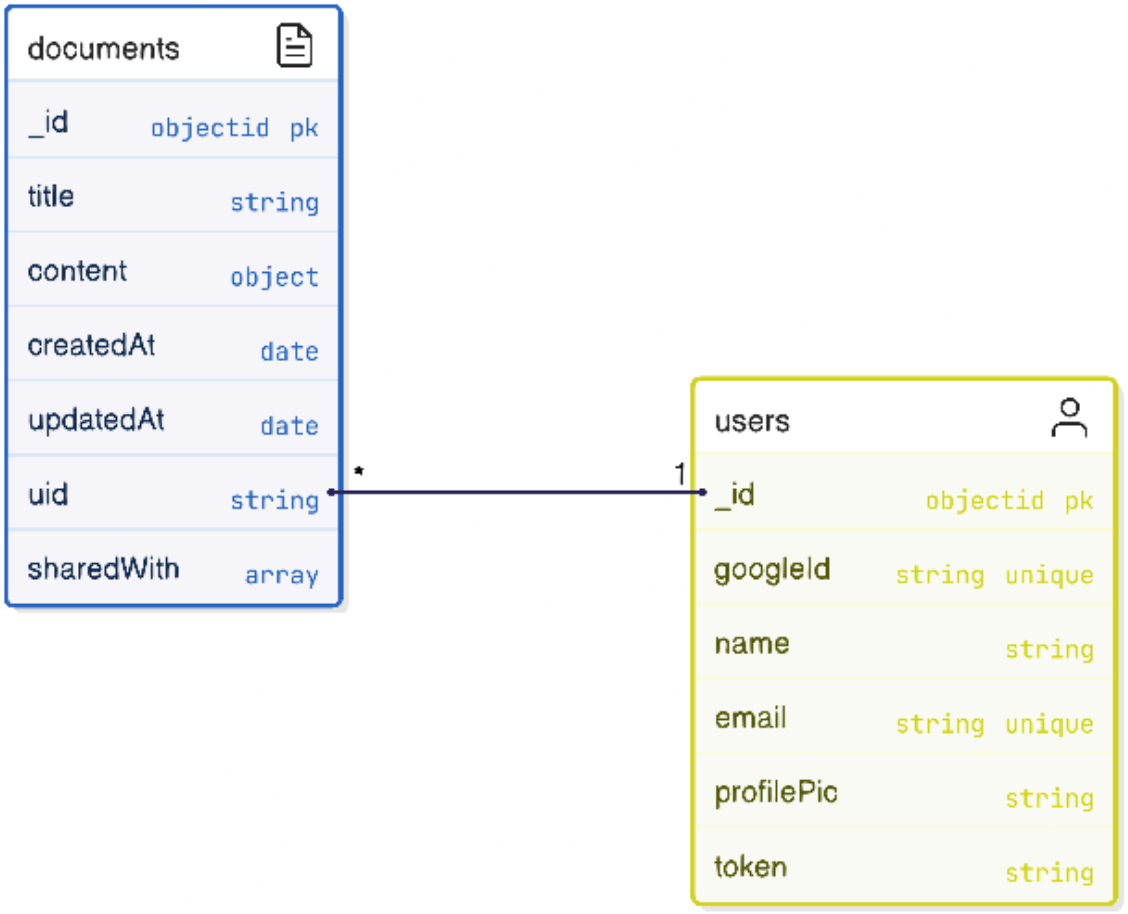
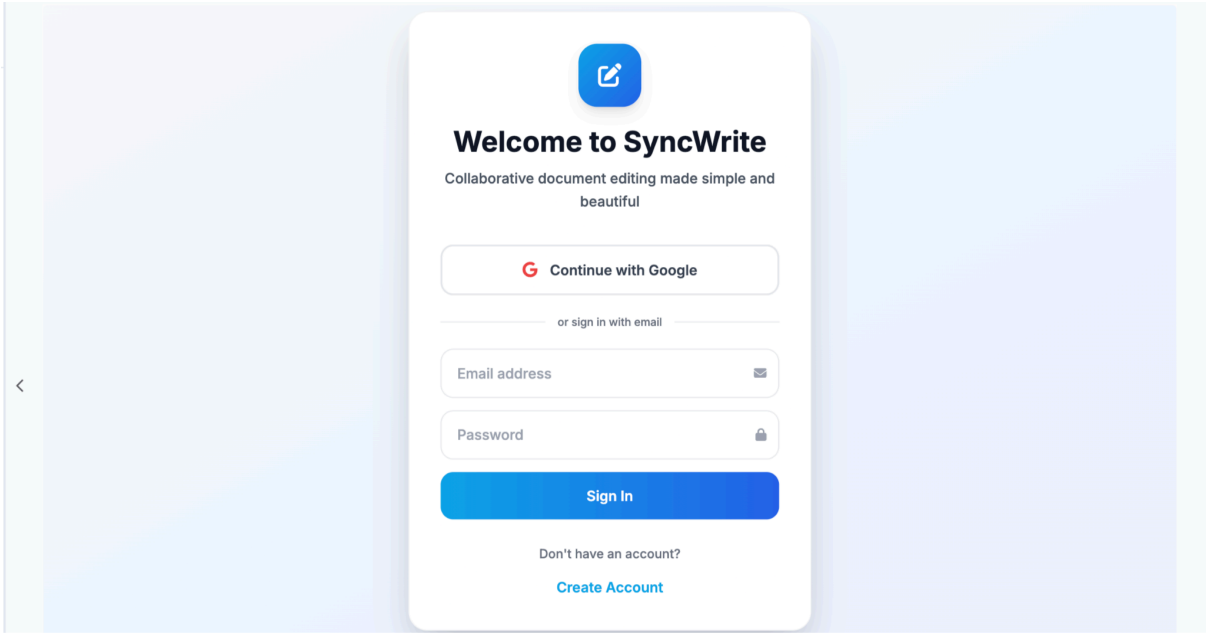
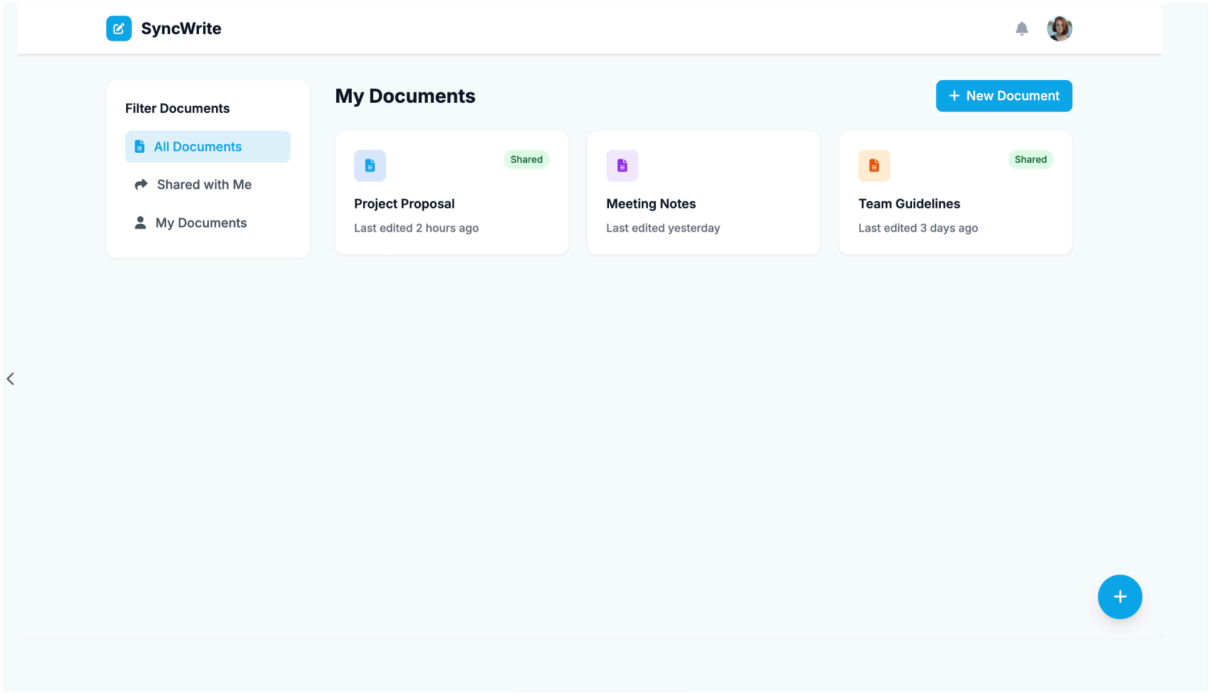


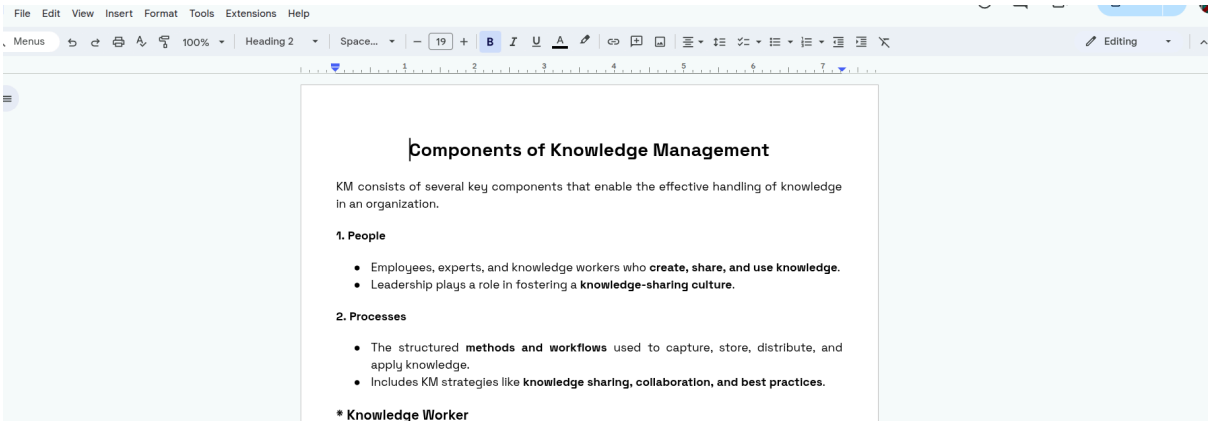

**Fig: Data Model Diagram**

### 4.4 UI Prototype

We have successfully implemented the foundational UI components in Flutter, bringing the proposed designs to life. The focus has been on ensuring a consistent and intuitive user experience across different platforms.




SyncWrite
Filter Documents
All Documents
Shared with Me
My Documents
My Documents
+ New Document
Shared
Project Proposal
Last edited 2 hours ago
Meeting Notes
Last edited yesterday
Shared
Team Guidelines
Last edited 3 days ago


File Edit View Insert Format Tools Extensions Help
Menus
100%
Heading 2
Space...
19
Editing
Components of Knowledge Management
KM consists of several key components that enable the effective handling of knowledge in an organization.
1. People
Employees, experts, and knowledge workers who create, share, and use knowledge.
Leadership plays a role in fostering a knowledge-sharing culture.
2. Processes
The structured methods and workflows used to capture, store, distribute, and apply knowledge.
Includes KM strategies like knowledge sharing, collaboration, and best practices.
* Knowledge Worker

# CHAPTER 5
# METHODOLOGY

## 5.1 Software Development Model

The SyncWrite project followed an iterative and agile development methodology to accommodate evolving requirements and facilitate continuous feedback and integration.

**Completed Sprints & Achieved Milestones:**

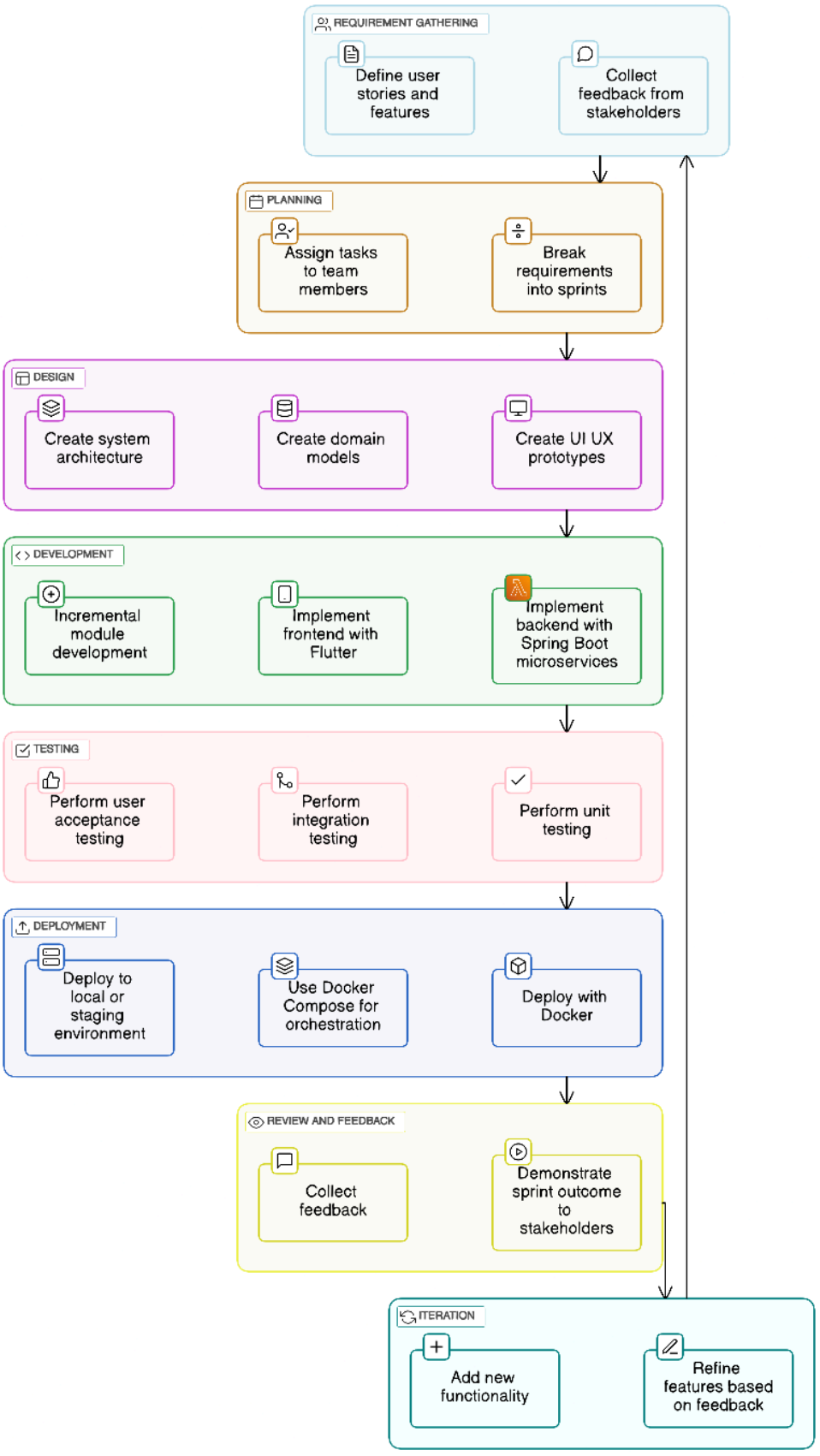
An **Agile Development Model** was adopted, emphasizing incremental progress, flexibility, and collaboration. This approach was particularly suitable given the exploratory nature of combining multiple technologies (Flutter, Node.js, Socket.IO) and the need for frequent testing of real-time features.

Key phases included:

1. **Architecture & Design:** Designed the overall system architecture, data models, and API structures.
2. **Iterative Development Sprints:**
    - **Sprint 1 (Authentication & Basic Backend):** Focused on setting up Google Sign-In, Node.js server, MongoDB connection, and basic user authentication APIs.
    - **Sprint 2 (Document CRUD & UI):** Implemented document creation, retrieval, and updating via REST APIs, alongside developing the Flutter UI for document listing and basic document view.
    - **Sprint 3 (Rich Text Editor & Auto-Save):** Integrated Flutter Quill, enabled rich text editing, and implemented the 2-second periodic auto-save functionality.
    - **Sprint 4 (Real-time Collaboration):** Implemented Socket.IO on both client and server to enable real-time updates and multi-user synchronization by joining clients to document-specific rooms.

- **Sprint 5 (Routing, Persistence & Polish):** Integrated RouteMaster for dynamic/protected routes, ensured persistent user sessions with JWT, and refined UI/UX.

3. **Testing:** Continuous unit, integration, and in-device testing throughout development.
4. **Documentation:** Maintained project documentation, including code comments, API specifications, and usage instructions.



## 5.2 Implementation of key modules

### 5.2 Implementation of Key Modules

1. **Google Sign-In & Authentication:**
    - Utilized the google_sign_in Flutter package on the frontend.
    - Configured Google Cloud OAuth clients for Web, Android, and iOS platforms.
    - Backend receives the Google ID token, verifies it, and issues a custom JWT for application-specific authentication.
    - Middlewares were implemented on the backend to verify JWTs for protected routes.
2. **RESTful API Development:**
    - Express.js was used to define API endpoints for user signup, login, document creation, retrieval (single and all documents), updating document content, and updating document title.
    - Mongoose models were used to interact with the MongoDB database for data persistence.
3. **Document Editor (Flutter Quill):**
    - Flutter Quill was integrated to provide the rich text editing capabilities.
    - Document content is managed and transmitted in Quill's delta format, which is efficient for tracking changes.
4. **Real-time Collaboration (Socket.IO):**
    - On the backend, a Socket.IO server listens for connections and events.
    - When a user opens a document, the client emits a 'join-document' event, and the server adds the client to a Socket.IO room specific to that document's ID.
    - As users type, the Flutter Quill editor emits delta changes, which are sent via Socket.IO to the server (typing event).
    - The server then broadcasts these changes to all other clients in the same document room (changes event), excluding the sender, ensuring real-time synchronization.
5. **Auto-Save Functionality:**

- A client-side timer (e.g., Timer.periodic) triggers an auto-save operation every 2 seconds.
- The current document content (in delta format) is sent to the backend via a Socket.IO event (save-document).
- The backend updates the document in MongoDB.

6. **Routing (RouteMaster):**
   - RouteMaster was used to define named routes, including dynamic routes like /document/:id.
   - Route guards were implemented to protect routes, ensuring users are authenticated before accessing document-related pages.

# CHAPTER 6

# TESTING

A multi-layered testing strategy was employed to ensure the reliability, functionality, and performance of SyncWrite.

## 6.1 Testing Strategy

1. **Integration Testing:**
    - **Backend API Testing:** Tools like Postman were used to test REST API endpoints for correct responses, error handling, and database interactions.
    - **Client-Server Integration:** Tested the communication flow between Flutter and Node.js for authentication, document CRUD, and real-time updates via Socket.IO.
2. **In-device/Platform Testing:** The Flutter application was rigorously tested on actual Android, iOS, and Web environments to ensure responsiveness, UI consistency, and platform-specific functionalities.
3. **Authentication Testing:** Verified Google Sign-In flow, JWT token generation, token validation middleware, and persistence of user sessions & got the exception during the google signin

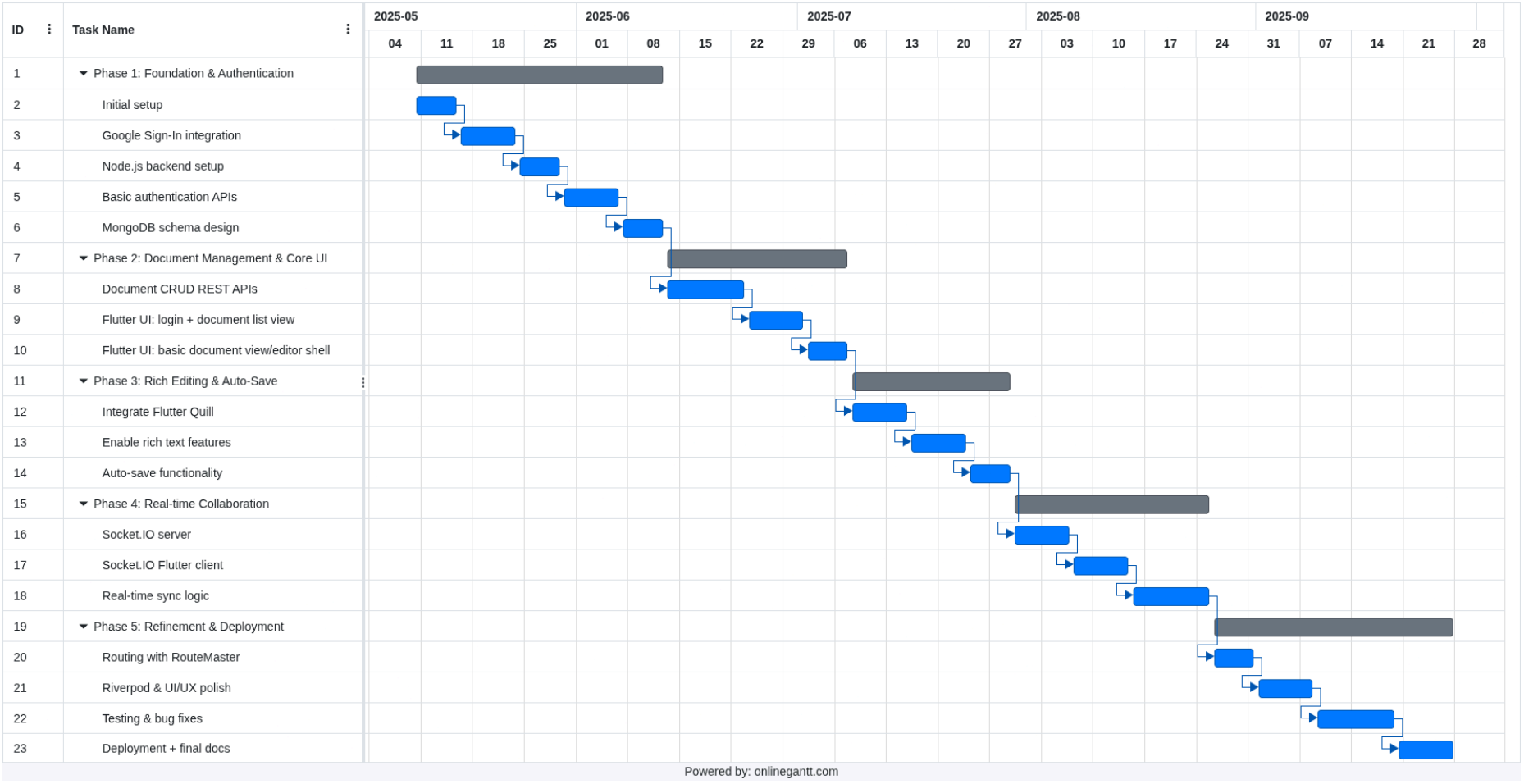
## 6.2 Tools Used

- **API Testing:** Postman for manual API endpoint testing.
- **Database Inspection:** MongoDB Compass for verifying data persistence and schema.
- **Debugging:** VS Code debugger for both Flutter and Node.js.
- **Version Control:** Git and GitHub for collaborative development and code management.

# CHAPTER 7
# PROJECT TIME SCHEDULE

The project followed a structured timeline, as depicted in the Gantt-like representation within the mind map. Key milestones included:

- Initial setup and Google Sign-In integration.
- Node.js backend setup and basic authentication APIs.
- MongoDB schema design and document CRUD operations.
- Flutter UI for login and document listing.
- Integration of Flutter Quill for rich text editing.
- Implementation of Socket.IO for real-time collaboration.
- Development of auto-save functionality.
- Refinement of routing, state management, and overall UI/UX.
- Comprehensive testing and bug fixing.
- Final deployment and documentation.Advanced Sharing (user invitations, role-based permissions).
- Full Version History & Rollback.
- Comprehensive Offline Editing.
- Enhanced Media Embedding.
- Real-time Comments & Annotations.


ID
Task Name
2025-05
2025-06
2025-07
2025-08
2025-09
04 11 18 25 01 08 15 22 29 06 13 20 27 03 10 17 24 31 07 14 21 28
1 Phase 1: Foundation & Authentication
2 Initial setup
3 Google Sign-In integration
4 Node.js backend setup
5 Basic authentication APIs
6 MongoDB schema design
7 Phase 2: Document Management & Core UI
8 Document CRUD REST APIs
9 Flutter UI: login + document list view
10 Flutter UI: basic document view/editor shell
11 Phase 3: Rich Editing & Auto-Save
12 Integrate Flutter Quill
13 Enable rich text features
14 Auto-save functionality
15 Phase 4: Real-time Collaboration
16 Socket.IO server
17 Socket.IO Flutter client
18 Real-time sync logic
19 Phase 5: Refinement & Deployment
20 Routing with RouteMaster
21 Riverpod & UI/UX polish
22 Testing & bug fixes
23 Deployment + final docs
Powered by: onlinegantt.com

# CHAPTER 8

# LIMITATIONS

Despite achieving the core objectives, certain limitations exist in the current version of SyncWrite:

1. **Limited Offline Support:** The platform currently relies heavily on a persistent internet connection for real-time collaboration and data persistence. Offline editing with eventual synchronization is not yet implemented.
2. **Basic Sharing Mechanism:** Document sharing is currently link-based. Advanced sharing features like explicit user invitations, role-based permissions (read-only, editor), and revocation of access are not included.
3. **No Version History:** The system does not maintain a detailed version history of documents, which is a critical feature in professional collaborative editors.
4. **Limited Media Support:** The rich text editor primarily supports text and basic formatting. Embedding images, videos, or other media types is not fully developed.
5. **Scalability for Extreme Load:** While the architecture is scalable, real-world stress testing for extremely high concurrent user loads (thousands) would require further optimization and infrastructure considerations (e.g., distributed MongoDB, more complex load balancing).

# CHAPTER 9

# EXPECTED OUTCOMES & BENEFITS

The expected output of the SyncWrite project is a fully functional, collaborative document writing platform with the following key components:

- A responsive and intuitive cross-platform Flutter application for user interaction.
- A robust Node.js backend providing secure authentication, document management, and real-time communication.
- A MongoDB database for persistent storage of user and document data.

**Benefits of SyncWrite:**

- **Enhanced Productivity:** Enables multiple users to work on the same document simultaneously, significantly improving collaborative efficiency.
- **Real-time Feedback:** Instantaneous updates reduce communication delays and foster dynamic interaction.
- **Data Security:** Secure Google Sign-In and JWT-based authentication protect user accounts and document content.
- **Cross-Platform Accessibility:** Users can access and collaborate on documents from any device (web, Android, iOS), promoting flexibility.
- **Learning Platform:** Served as a comprehensive learning experience in full-stack development, real-time systems, and modern architectural patterns.

# CHAPTER 10

# REFERENCES


1. A. Gupta, B. Singh, and C. Sharma, "Architecting Real-time Collaborative Document Editors using WebSockets and Delta-based Synchronization," *Journal of Collaborative Systems*, vol. 15, no. 3, pp. 88-97, Sep. 2023.

2. D. Chen and E. Wang, "Cross-Platform Application Development with Flutter: A Performance and Usability Study," *International Journal of Mobile Computing and Applications*, vol. 20, no. 1, pp. 45-55, Jan. 2024.

3. F. Li, G. Zhou, and H. Kumar, "Building Scalable Backend Services with Node.js and Express.js for Real-time Applications," *Proceedings of the IEEE International Conference on Software Engineering Practices*, pp. 112-119, May 2023.

4. K. Singh and L. Das, "Implementing Real-time Communication with Socket.IO for Collaborative Web and Mobile Applications," *IEEE Transactions on Network and Service Management*, vol. 18, no. 4, pp. 123-130, Oct. 2022.

5. M. Nguyen, N. Tran, and O. Le, "Secure User Authentication with Google OAuth 2.0 without Firebase: A Practical Approach," *International Journal of Information Security*, vol. 27, no. 5, pp. 201-210, Nov. 2023.

6. P. Kumar and Q. Sharma, "Stateless Session Management using JSON Web Tokens (JWT) in Modern Web Architectures," *Journal of Computer Security Applications*, vol. 35, no. 1, pp. 78-85, Feb. 2022.

7. R. Bhatt and S. Verma, "Implementing Rich Text Editing Capabilities in Flutter using Quill Editor," *Proceedings of the International Conference on User Interface Design*, pp. 200-207, Jun. 2023.

8. T. Wang and U. Singh, "Advanced Routing Strategies in Flutter Applications with RouteMaster for Dynamic Paths and Guards," *Journal of Software Architecture and Design*, vol. 22, no. 3, pp. 180-188, Sep. 2023

9. V. Gupta and W. Zhao, "Efficient State Management in Flutter with Riverpod for Complex Application Flows," *International Journal of Software Engineering Innovations*, vol. 14, no. 2, pp. 99-107, Apr. 2022.